

make INSTALL_MOD_STRIP=1 modules_install

Next, run a *make install*.

Updating with *initramfs*

Run the command, *sudo update-initramfs -c -k 2.6.32-dips+*

For readers who are keen to know what *initramfs* is, it is a root filesystem that is loaded at an early stage of the boot process. It is the successor of the old *initrd*.

Verify the installation

Now, we are almost done! To verify, check your */boot* directory. Are you able to find the new kernel image and the *config* file for your build? If yes, then congratulations! You have succeeded in building your kernel.

Modify the GRUB file and reboot

GRUB stands for Grand Unified Bootloader. It is a boot-loader package from GNU. GRUB provides a user the choice to boot any one of multiple operating systems installed on a computer, or to select a specific kernel configuration available on a particular operating system's partitions.

You can find the GRUB file at */boot/grub/grub.cfg*, as shown in Figure 2.

Issue the command *sudo update-grub* and your GRUB file will be modified. You should be able to see an entry for your kernel, which looks like what's shown below:

```
### BEGIN /etc/grub.d/05_debian_theme ###
set menu_color_normal=white/black
set menu_color_highlight=black/light-gray
### END /etc/grub.d/05_debian_theme ###

### BEGIN /etc/grub.d/10_linux ###
menuentry 'Ubuntu, with Linux 2.6.32-dips+' --class ubuntu
--class gnu-linux --class gnu --class os {
    recordfail
```

```
deep@linuxdeep@linux-laptop:~$ git clone git://git.kernel.org/pub/scm/linux/
kernel/git/torvalds/linux-2.6.git linux-2.6
```

Figure 1: Download the kernel source

```
deep@linuxdeep@linux-laptop:~$ cat /boot/grub/grub.cfg
#
# DO NOT EDIT THIS FILE
#
# It is automatically generated by /usr/sbin/grub-mkconfig using templates
# from /etc/grub.d and settings from /etc/default/grub
#

### BEGIN /etc/grub.d/40_header ###
if [ -s $prefix/grubenv ]; then
    load_env
fi
set default="0"
if [ $(prev_saved_entry) ]; then
    set saved_entry=$(prev_saved_entry)
    save_env saved_entry
    set prev_saved_entry=
    save_env prev_saved_entry
    set boot_once=true
fi

function savedefault {
    if [ -z $boot_once ]; then
```

Figure 2: GRUB file

```
insmod ext2
set root='(hd0,5)'
search --no-floppy --fs-uuid --set 0b0be63-24f6-45c0-a3df-
5d9c447b5a09
linux /vmlinuz-2.6.32-dips+ root=UUID=325f1d1d-2070-45f3-
94ed-2027c5ffcbf3 ro quiet splash
initrd /initrd.img-2.6.32-dips+
}
```

Now, just reboot the system. Your system should now boot up with your just-built kernel. You can verify it by using the *uname -a* command again.

That's all you need to build a kernel. Now wasn't that simple? So, have fun building your kernel! **END** 

By: Deepti Sharma

The author is an open source enthusiast with almost a decade of experience in the design, development and quality validation of systems software tools (compilers/assemblers/device drivers) for embedded systems. She loves writing for LFY. She can be reached at deepti.acmet@gmail.com.

EFY Group's New Initiative

EB TIMES

Will be launched on Sept 11, 2012

This 8-page monthly **B2B Newspaper** will be a resource for traders, distributors, dealers, and those who head channel business, as it aims to give an impetus to channel sales

